TRABAJO PRÁCTICO – UNIDAD 5

ACTIVIDAD EN EQUIPO

FECHA DE ENTREGA: MARTES 27 DE OCTUBRE

Comisión Tarde: hasta las 13.00 hs.

Comisión Noche: Hasta las 18.00 hs.

La entrega se realizará en el aula virtual cuando esté en funcionamiento.

PARTE 2

- Seleccionar una funcionalidad (diferente al TP de la Unidad 4)
- Seleccionar un requerimiento que permita realizar una prueba de verificación.
- Para el requerimiento seleccionado, elaborar un Caso de prueba.

Campo de caso de prueba	Descripción
ID de caso de prueba	TC-DEP-01
Requerimiento	RF-AL-03: El sistema debe permitir al alumno inscribirse a actividades deportivas (HU-01). Criterio verificado: no se permite superar dos deportes activos.
Prueba diseñada por	Hakanson, Ian y Morales, Elias
Fecha de diseño de la prueba	27/10/2026
Prueba ejecutada por	Morales, Elias (manual y automatizada con Cypress)
Fecha de ejecución de la prueba	27/10/2026
Condición previa	API, base de datos y frontend en funcionamiento con los datos demo cargados (database/seed.sql). El alumno ana.alumna (contraseña Educar2027!) tiene una inscripción activa a Futbol (grupo 1, lunes 16:00-17:00) y no está inscripta a ningún otro deporte.
Pasos de prueba	Resultado esperado
1. Iniciar sesión como Alumno (usuario ana.alumna).	Se accede al panel del alumno. 'Mis inscripciones' muestra Futbol y el contador indica 1/2.
2. Ingresar al 'Catálogo deportivo'.	Se listan los grupos disponibles con deporte, horario y docente

	responsable; los grupos sin conflicto muestran el botón 'Inscribirme'.
3. Inscribirse a Ajedrez (miércoles 16:00 a 17:00).	La inscripción se confirma con deporte, grupo, horario y docente. Ajedrez aparece en 'Mis inscripciones' y el contador pasa a 2/2.
4. Volver a revisar el catálogo deportivo.	Todos los botones restantes (por ejemplo Atletismo) muestran 'Límite alcanzado' y están deshabilitados.
5. Forzar un tercer deporte saltando la interfaz: POST /api/student/enrollments con groupId 3 (Atletismo) y el token del alumno.	La API responde 409 con el código SPORT_LIMIT_REACHED y el mensaje 'El límite máximo de dos deportes ya fue alcanzado'. No se crea la inscripción; el alumno sigue con 2 deportes.

Resultado de la ejecución: APROBADO. El caso se automatizó en cypress/e2e/inscripcion-deportes.cy.ts y se ejecutó contra la aplicación real (frontend + API + PostgreSQL): 2 pruebas aprobadas. La regla también está cubierta por las pruebas unitarias de la API (Vitest): 'rejects a third sport'.

- **Prueba estática:** *Revisión por pares (Peer Review):* Compañeros evalúan el trabajo para detectar fallos.
    - Indicar que estrategias de buenas prácticas debería cumplir el código para poder agilizar la evaluación del mismo.
    - Indicar estrategias para poder aplicar la técnica indicada.

Respuesta – Buenas prácticas del código para agilizar la revisión

• Pull Requests pequeños y enfocados (una rebanada vertical por vez, idealmente menos de 400 líneas): se revisan más rápido y con más atención.

• Convenciones únicas y automatizadas: ESLint define las reglas y el estilo del repositorio (comillas dobles, punto y coma, 2 espacios) para que la revisión no discuta formato.

• Nombres descriptivos en inglés para identificadores y comentarios en español donde aporten el porqué, no el qué.

• Separación MVC en tres capas (rutas en app.ts, lógica en *.service.ts, vista en apps/web) y funciones cortas con una sola responsabilidad.

• Errores explícitos con AppError(statusCode, mensaje, código) en lugar de valores mágicos; así el revisor ve de inmediato la regla de negocio.

• Pruebas junto al cambio: cada regla de negocio nueva viene con su test; el revisor lee el test para entender la intención.

• Commits atómicos con Conventional Commits (feat:, fix:, docs:, chore:) y una plantilla de PR con checklist (.github/pull_request_template.md).

• Sin código muerto ni imports sin usar (la regla no-unused-vars de ESLint los detecta) y sin secretos en el repositorio.

Respuesta – Estrategias para aplicar la revisión por pares

• Todo cambio entra a main por Pull Request: no se hace push directo (branch protection de GitHub).

• Mínimo 1 aprobación de otro integrante distinto del autor antes de mergear; el archivo .github/CODEOWNERS asigna revisor automáticamente.

• El PR solo puede mergearse si pasan los checks automáticos: lint, typecheck, tests, build, E2E y Quality Gate de SonarCloud.

• El revisor usa una lista de verificación: ¿cumple el criterio de aceptación de la HU?, ¿hay tests?, ¿respeta las capas?, ¿maneja errores?, ¿expone datos de otro usuario?

• Comentarios en línea sobre el diff de GitHub, resueltos antes de aprobar; el autor responde con commits adicionales.

- Rotación de revisores entre Ian y Elias para repartir el conocimiento del código.
    - **Para el día martes 27 debe estar disponible :**
        - **En un documento Word la siguiente información:**
            - Historia de usuario
            - Arquitectura de la aplicación
            - Información adicional que considere necesaria.
        - Configurar un ***linter*** específico para su lenguaje directamente en su editor de código (IDE).
        - Configurar ***Pre-commit Hooks***: Implementar herramientas como **Husky** para bloquear los *commits* locales si el código no supera las reglas básicas de formato y sintaxis.
        - Políticas de ***Pull Requests*** (PR)
        - Orquestador Centralizado (Recomendado: SonarQube / SonarCloud)

Respuesta – Linter en el editor (ESLint)

Se instaló ESLint 9 con typescript-eslint y eslint-plugin-react-hooks, configurado en eslint.config.mjs (flat config) para toda la monorepo (API Node y frontend React). Reglas principales: no-unused-vars, eqeqeq, prefer-const, no-var, react-hooks/rules-of-hooks. En Visual Studio Code se usa la extensión 'ESLint' (dbaeumer.vscode-eslint) con eslint.useFlatConfig=true y editor.codeActionsOnSave -> source.fixAll.eslint, de modo que los errores se subrayan mientras se escribe y se corrigen al guardar. Comandos: npm run lint y npm run lint:fix. Estado actual: 0 errores y 12 advertencias (react-hooks/exhaustive-deps).

Respuesta – Pre-commit Hooks (Husky + lint-staged)

Se instaló Husky 9 y lint-staged. El hook .husky/pre-commit ejecuta 'npx lint-staged' (ESLint solo sobre los archivos .ts, .tsx y .mjs preparados para el commit) y luego 'npm run typecheck'. Si hay un error de lint o de tipos, el commit se bloquea y no se crea. El script 'prepare' (husky) instala los hooks automáticamente al hacer npm install.

Respuesta – Políticas de Pull Requests

- Rama protegida main: PR obligatorio, 1 aprobación mínima, rama actualizada contra main y conversaciones resueltas.
- Checks obligatorios: workflow CI (lint, typecheck, test, build), job E2E con Cypress y SonarCloud Quality Gate.
- Plantilla de PR con descripción, tipo de cambio y checklist de calidad (.github/pull_request_template.md).
- CODEOWNERS para solicitar revisión automática (.github/CODEOWNERS) y mensajes de commit en formato Conventional Commits.
- Detalle completo en docs/POLITICA_PR.md. Las reglas de protección de rama se activan desde Settings > Branches del repositorio en GitHub.

Respuesta – Orquestador centralizado (SonarCloud)

Se agregó sonar-project.properties (fuentes, pruebas y reporte de cobertura lcov) y el workflow .github/workflows/sonarcloud.yml, que corre las pruebas con cobertura y envía el análisis a SonarCloud en cada push y Pull Request a main. Estado: la configuración está en el repositorio; para activar el escaneo falta vincular el repositorio en sonarcloud.io y cargar el secreto SONAR_TOKEN en GitHub (sin el secreto el paso se omite).

- **Prueba Dinámica**
    - Configurar SELENIUM o CYPRESS para realizar **pruebas de software (Quality Assurance o QA)** en aplicaciones web, permitiendo simular las acciones de un usuario real.
    - Ejemplificar una prueba de regresión.
    - De acuerdo a la tecnología que utilizó para desarrollar su aplicación seleccionar una herramienta para realizar pruebas automatizadas y configurarla para aplicarla.

Respuesta – Configuración de Cypress

La aplicación es web (React + Vite) con una API Node/Express, por eso se eligió Cypress, que corre en JavaScript/TypeScript sobre el mismo stack. Se instaló cypress 13 como dependencia de desarrollo. Archivos: cypress.config.ts (baseUrl http://localhost:5173 y URL de la API), cypress/support/e2e.ts (comandos reutilizables apiLogin, uiLogin y resetSportEnrollments) y cypress/e2e/*.cy.ts. Se ejecutan con 'npm run e2e' (modo consola) o 'npm run e2e:open' (modo interactivo).

Las pruebas simulan a un usuario real: abren el login, escriben usuario y contraseña, hacen clic en 'Inscribirme' y verifican en pantalla que aparezca la inscripción y que el catálogo quede bloqueado ('Límite alcanzado'). Además consultan la API directamente para comprobar que la regla no depende de la interfaz. El job 'e2e' de GitHub Actions las corre en cada push contra un PostgreSQL de servicio.

Ejecución local: 2 archivos de prueba, 4 pruebas, 4 aprobadas (inscripcion-deportes.cy.ts y regresion-horarios.cy.ts).

Respuesta – Ejemplo de prueba de regresión

Defecto corregido en el commit 'fix: reject overlapping schedules within the same course': se permitían horarios superpuestos. Para que no reaparezca, la prueba cypress/e2e/regresion-horarios.cy.ts se ejecuta en cada cambio:

- Se inscribe a ana.alumna en Futbol (lunes 16:00-17:00) y se intenta Natación (lunes 16:30-17:30): la API debe responder 409 SCHEDULE_CONFLICT.
- Control positivo: un grupo adyacente sin superposición (Ajedrez, miércoles) debe aceptarse.

Si en el futuro un cambio rompe la validación de horarios, el CI falla y el PR no puede mergearse. Es una regresión porque protege una funcionalidad ya corregida de cambios nuevos.

Respuesta – Herramienta de pruebas automatizadas según la tecnología

API (Node + TypeScript): Vitest + supertest + pg-mem (50 pruebas aprobadas, base PostgreSQL en memoria, sin servicios externos). Frontend y flujo completo: Cypress. Ambas corren en el pipeline de GitHub Actions (npm test y npm run e2e).

- **Métricas**
  - Seleccionar dos métricas e indicar como evaluaría su rendimiento. Puede usar una herramienta o verificar si las herramientas aplicadas en puntos anteriores proporcionan información sobre métricas.

Respuesta – Métrica 1: cobertura de código

Herramienta: Vitest con el proveedor V8 (npm run test:coverage, genera apps/api/coverage/lcov.info) y SonarCloud como visor central. Resultado actual de la API: 84,61 % de líneas, 73,21 % de ramas y 92,07 % de funciones; el servicio de inscripciones deportivas (enrollment.service.ts) tiene 234 de 244 líneas cubiertas (95,9 %). Criterio de evaluación: objetivo mínimo de 80 % de líneas en código nuevo; si baja de ese valor el Quality Gate falla y el PR se bloquea. Se revisa en cada PR comparando contra la ejecución anterior.

Respuesta – Métrica 2: calidad estática (errores y advertencias del linter)

Herramienta: ESLint (npm run lint), integrado al hook pre-commit, al CI y a SonarCloud. Resultado actual: 0 errores y 12 advertencias (todas react-hooks/exhaustive-deps) sobre unas 8.700 líneas de la API y las páginas del frontend. Criterio de evaluación: 0 errores como condición obligatoria para commitear y mergear, y una tendencia de advertencias que no crezca de un PR a otro (tope configurado en lint-staged). Se complementa con la tasa de pruebas aprobadas: 50/50 pruebas unitarias y 4/4 pruebas E2E (100 %).

Historia de usuario:

HU-01 – Inscripción a deportes

Como Alumno, quiero inscribirme a actividades deportivas para participar en propuestas extracurriculares.

Criterios de aceptación:

- Se muestra el catálogo de deportes y grupos disponibles.
- No se permite superar dos deportes activos.
- No se permite seleccionar horarios superpuestos.
- La inscripción confirma deporte, grupo, horario y docente responsable.

Requerimiento asociado: RF-AL-03. Caso de uso: CU-01 (Inscribirse a un deporte). Estado: Done (Sprint 1).

Arquitectura de la aplicación

Sistema de gestión 'Educar para Transformar'. Monorepo con tres capas (MVC) y base de datos relacional:

Capa	Tecnología	Ubicación	Responsabilidad
Vista (frontend)	React 18 + TypeScript + Vite + React Router	apps/web	Pantallas por rol (alumno, padre, docente, administrador, director); todas las llamadas HTTP pasan por src/api.ts.
Controlador (API)	Node 20 + Express + TypeScript (ESM)	apps/api/src/app.ts	Define las rutas REST, valida entradas y traduce AppError a respuestas HTTP.
Modelo (servicios)	TypeScript + pg	apps/api/src/*.service.ts	Lógica de negocio y acceso a datos; transacciones con withTransaction; reglas como el límite de dos deportes en enrollment.service.ts.

Seguridad	JWT + bcrypt + RBAC	apps/api/src/auth.ts	requireAuth / requireRoles; la API revalida al usuario en cada request.
Datos	PostgreSQL 16	database/schema.sql, seed.sql, migrations/	Esquema normalizado, datos demo y migraciones incrementales.

Patrones de diseño de la consigna: Singleton (database.ts), Proxy (auditProxy.ts, autoriza y audita) e Iterator (rowIterator.ts). Flujo de la HU-01: Navegador (React) -> POST /api/student/enrollments con JWT -> controlador en app.ts -> enrollment.service.ts (valida límite, duplicados y superposición de horarios dentro de una transacción) -> PostgreSQL -> respuesta 201 o error 409 con código de negocio.

Información adicional

- Repositorio: github.com/kahnaisehC/educar-para-transformar (rama rama_nueva; la protección de main por Pull Request se activa en Settings > Branches).
- Integración y despliegue continuos: GitHub Actions (lint, typecheck, test, build, E2E y SonarCloud) y despliegue en Render (API, sitio estático y PostgreSQL) definido en render.yaml.
- Herramientas instaladas en esta entrega: ESLint, typescript-eslint, Husky, lint-staged, Cypress, wait-on, @vitest/coverage-v8 y la configuración de SonarCloud.
- Todos los archivos de esta entrega (documento, PDF y guía de configuración) están en la carpeta unidad5v2 del repositorio; el detalle de uso está en unidad5v2/README.md.
- Integrantes: Hakanson, Ian y Morales, Elias.